

Beginning Python for financial data

Machine Learning in Finance · Session 1

Welcome

MACHINE LEARNING IN FINANCE · SESSION 1

Programming and machine learning for people who have never written a line of code. We start from zero.

Who is teaching this

THE ACADEMIC PART

- **Jonas Theill Bøjstrup**, PhD student
- Research: **image-based prediction in finance** and deep learning
- Teaching this course for the **second time**
- Supervised **6+** master's theses

THE REST OF ME

- Art collector
- Video games
- Becoming a **dad in December**

Questions any time, in class or by email to jobo@econ.au.dk, whoever is lecturing. **Bezirgen Veliyev** lectures some of the sessions.

Why Python?

Everyone uses it

The dominant language in data science and quantitative finance. What you learn transfers to a real job.

Readable

Python code reads close to plain English. That matters enormously when you are starting out.

Batteries included

Free libraries for data, statistics, and machine learning: pandas, scikit-learn, and many more.

Endless help

A vast community. Almost any error you hit has been asked and answered somewhere already.

Free, open, and runs anywhere: later in the course, also on your own laptop with no internet.

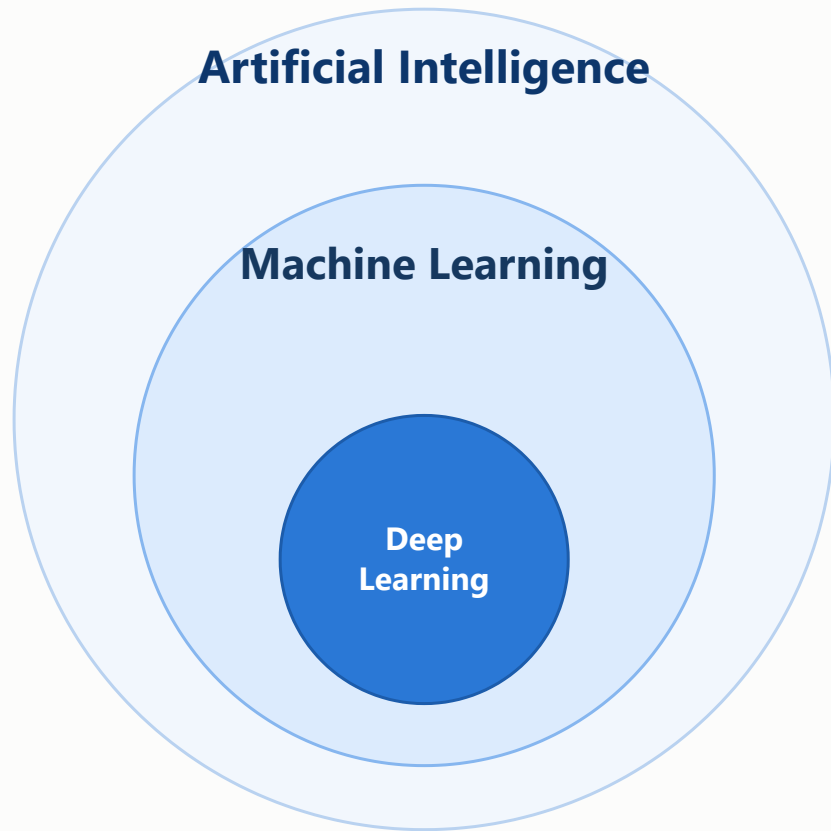
Why learn machine learning?

Finance has become a **data discipline**.

- Asset management, risk, trading, credit, fintech: all increasingly run on models fit to data.
- Job postings that used to ask for “econometrics” now ask for **machine learning**.
- You already know one such model: **ordinary least squares**. This course extends that instinct to more flexible tools.

The goal is not to make you a computer scientist. It is to let you **read, build, and judge** the models that finance now runs on.

What even is machine learning?



Machine learning

a model that learns patterns from data, instead of being given explicit rules.

Deep learning is one family of ML methods; ML is one route to AI.

Your OLS regression is already a simple machine-learning model: it *learns* its coefficients from data. We build outward from there.

Where this course is going

1 to 3	Python: variables, loops, functions and dictionaries; NumPy, pandas and matplotlib	from today
4	foundations of machine learning: features, targets, splits and scores	no model yet
5 and 6	model selection and cross-validation; ridge and lasso	the first models
7	a house in Aarhus, with every tool so far	a case lecture
8 and 9	classification: logistic regression, thresholds and k-NN	labels
10	tomorrow's power price in West Denmark	a case lecture
11 and 12	trees, forests and boosted trees	flexible models
13	the two cases with trees, and the exam	the wrap-up

Thirteen sessions, from the week of **31 August 2026** (week 36) to week 42.

How the course works

IN CLASS

Lectures

We work through concepts together, with live code you can see running. This is where ideas are introduced.

AT HOME

Exercises

A notebook of short practice tasks each week, with hints and solutions folded in. This is where the learning sticks.

AT HOME

The case

One financial investigation that grows through the course, a part after most sessions. You carry it forward each week.

Lectures are for watching and understanding. **You learn to code by doing the exercises and the case yourself, between sessions.**

This is a demanding course



Going from **no programming** to machine learning in seven weeks is a steep climb. Be ready for it.

- I can explain ideas and show code, but **watching is not learning to code**.
- You become fluent by **doing it yourself**. The exercises and case are where that happens.

My job is to teach the concepts and point the way. **Your job is to do the work.**

Using AI on this course

AI assistants are useful and are not banned. Use them **to learn**, not to avoid learning.

Helpful

- ✓ Explain an error message you do not understand
- ✓ Explain a concept a different way
- ✓ Suggest what a line of code does
- ✓ Generate extra practice problems

Harmful

- ✗ Pasting in solutions you cannot reproduce
- ✗ Submitting code you do not understand
- ✗ Leaning on it instead of building skill
- ✗ Relying on it for the exam: you cannot

A good test: if you could not rewrite it from memory afterwards, you have not learned it yet.

The exam

3 hours

on site, at a fixed time

No internet

no AI, no web search, no Colab

Your laptop

Python running locally

The exam runs **on your own machine, offline**. We begin in Google Colab because it is the easiest place to start, and later move to a local setup: a **VS Code tutorial comes in Session 3**.

Because the exam is offline, the skill has to be **yours**. That is the whole reason the exercises matter.

Today

By the end of this session you can:

- run Python in a **notebook** and say what the *runtime* remembers
- store values in **variables** and use the basic **types** of data
- hold many prices in a **list** and pick out any day or window
- compute a **simple return** from two prices
- read a Python **error message** calmly

And you meet the course **case**: a four-part risk report on real US stocks.

1 • Notebooks

PART 1

Where our code lives, and how a code cell shows its result.

A notebook is a stack of cells

text	Compute this week's return
[]	$(248.83 - 183.56) / 183.56$
0.3556	

A **notebook** is a document read top to bottom, made of two kinds of cell. The result of a code cell appears **directly beneath it**.

Two kinds of cell

- **Text cells** hold explanation: headings, sentences, the story.
- **Code cells** hold Python, and can be **run**. Their output appears below.

That is the whole idea. A lecture, a calculation, and its result can live in one scrollable document, which is exactly what these slides are.

We begin in **Google Colab**, which runs notebooks in your browser with nothing to install. You need only a Google account.

Running a cell

Click into a code cell and press **Ctrl + Enter**. The cell runs and shows the value of its **last line**, and only the last one.

These slides run Python too. Run this: just one number appears.

Python Code

183.56
248.83

248.83

A real Python cell, running in your browser. Change the numbers and run it again. Clicking **Run** does exactly the same thing.

The keys that run a cell

key	what it does	where
Ctrl + Enter	runs the cell, stays put	everywhere, including these slides
Shift + Enter	runs the cell, moves to the next	Colab, Jupyter, VS Code

On a Mac, use **Cmd** in place of **Ctrl**.

i **Shift + Enter** is the habit most people settle into in Colab. The little editor built into these slides is the one place it does nothing at all, so here use **Ctrl + Enter** or the **Run** button.

Showing more than the last line

To show **several** values, use `print(...)`:

Python Code

```
print(183.56)  
print(248.83)
```

```
183.56  
248.83
```

`print` is our first **function**: a named tool you use by writing its name and round brackets. It displays whatever you put inside, each on its own line. We will lean on it constantly.

2 • Python as a calculator

PART 2

Expressions, and leaving notes to yourself.

Expressions

An **expression** is code that produces a value. The text after a **#** is a **comment**: Python ignores it; it is a note for humans.

Python Code

```
120 * 45.50 # a comment: 120 shares at 45.50 each
```

5460.0

Change the numbers and run it. Arithmetic works as you would expect.

The arithmetic you have

write	meaning
<code>+ - * /</code>	add, subtract, multiply, divide
<code>**</code>	power: <code>1.05 ** 10</code> is ten years of 5% growth
<code>()</code>	grouping, exactly as in mathematics

Order of operations is the usual one: powers, then `*` and `/`, then `+` and `-`. Parentheses override it.

Powers: compound growth

1,000 invested at 4% for five years is a single expression:

Python Code

```
1000 * (1 + 0.04) ** 5
```

```
1216.6529024000001
```

Change the rate to **0.06** and run again. Compounding is just repeated multiplication, written with ******.

3 • Variables

PART 3

Giving a value a name, so you can use it again.

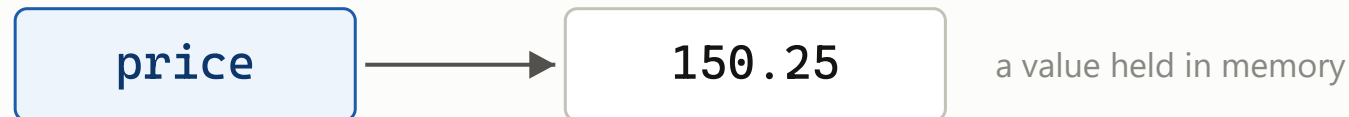
Variables: naming a value

A **variable** gives a value a name. The `=` sign means **assign**: “store this under that name”, not “equals”.

Python Code

```
price = 150.25  
shares = 40  
price * shares
```

6010.0



Once named, it stands for the value

Anywhere you write the name, Python uses the value:

Python Code

```
price = 150.25
shares = 40
holding_value = price * shares
holding_value
```

6010.0

`holding_value` now holds the result, ready to reuse, with no need to retype the multiplication.

Assignment can overwrite

Assigning to a name that already exists **replaces** its value. The old one is gone.

Read `price = price * 1.02` from the **right**:

take the current `price`, multiply by 1.02, and store the result back into `price`.

Overwrite in action

PREDICT

Line 2 uses `price` on **both sides** at once. What do you think that does, and what is `price` at the end? Reason it out before you run.

Python Code

```
price = 150.25
price = price * 1.02
price
```

Can a name really appear on both sides of `=`?

Naming rules

Python **requires**:

- letters, digits, and underscores only
- a name may **not start with a digit**
- no spaces: `share price` fails, `share_price` works
- case matters: `Price` and `price` are two different names

Naming: your judgment

Both are legal. Only one will make sense to you next month:

```
x = 0.0212  
weekly_return = 0.0212
```

Write names for the person who reads the code later. That person is usually **you**, a week from now.

Your turn 1

You bought 120 shares at 45.50 and sold at 48.10. Complete the cell to compute the **profit**.

Python Code

```
shares = 120
buy_price = 45.50
sell_price = 48.10

profit = ...
profit
```

The result should be a little over 300.

4 • The notebook's memory

PART 4

Now that you can make variables: the notebook remembers them between cells.

The runtime remembers (1)

Run this cell. It stores a value in memory under the name `portfolio_value`:

Python Code

```
portfolio_value = 100 * 150.25  
portfolio_value
```

15025.0

The value is now **held in memory**. This memory is called the **runtime**.

The runtime remembers (2)

A *later* cell can use what an earlier cell defined, without repeating it:

Python Code

```
portfolio_value * 1.10
```

16527.5

This works only because the cell above **ran first** and left `portfolio_value` in memory.

A notebook is not just text on a page. It has a **memory** that fills up as you run cells.

Order of execution matters

Cells can be run in **any order** you click them, and the runtime only knows what it has been *told so far*.

Run a page top to bottom and all is well. Jump around, and a cell may look for something that does not exist **yet**. The next two cells show it.

A cell that runs too early

PREDICT

Fresh runtime: nothing has defined `bonus` yet. **What do you think happens when you run this?** Talk it through before you press Run.

Python Code

```
bonus * 2
```

Does Python guess a value? Refuse? Something else?

Define it, then it works

Give **bonus** a value, and the very same expression works:

Python Code

```
bonus = 1200  
bonus * 2
```

2400

The error before was not a fault in the code: the runtime simply had **nothing stored** under that name. Define it first, and all is well.

The safe habit

When a notebook gets confusing, reset it: **Runtime** → **Restart and run all**.

A notebook is only trustworthy if it runs cleanly **top to bottom from a fresh start**.
We will hold to that all course.



Restarting clears the runtime's memory and re-runs every cell in order. It fixes almost every "but it worked a minute ago" mystery.

5 • Every value has a type

PART 5

Numbers, text, and true/false, and why the difference costs money.

Two kinds of number

Python tells you the type of any value with the `type(...)` function:

Python Code

```
print(type(40))  
print(type(150.25))
```

```
<class 'int'>  
<class 'float'>
```

- `int` for whole numbers: share counts, trading days
- `float` for numbers with a decimal point: prices, returns, rates

Division always gives a float

Even when it divides evenly:

Python Code

```
100 / 4
```

```
25.0
```

The result is `25.0`, a float, not `25`. Division in Python always produces a decimal number.

Text: strings

A **string** is text inside quotes, such as tickers, names and dates:

Python Code

```
ticker = "AAPL"  
print(ticker)  
print(type(ticker))
```

```
AAPL  
<class 'str'>
```

Gluing strings together

+ between strings joins them. It does **not** add:

Python Code

```
ticker = "AAPL"  
ticker + " is a US stock"
```

```
'AAPL is a US stock'
```



"150" in quotes is *text that looks like a number*. It is not a number, and that difference causes real bugs, because prices read from files often arrive as text.

Numbers in disguise

PREDICT

Both values are wrapped in quotes. **What will + do here**, and is it what you would want? Decide before you run.

Python Code

```
"10" + "5"
```

Talk to your neighbour first.

Numbers in disguise: the fix

`float(...)` turns text into a number. Now it really adds:

Python Code

```
float("10") + float("5")
```

15.0

No error is **not** the same as no mistake. `"10" + "5"` gave `"105"` with no complaint, and a silently wrong answer is the dangerous kind.

Showing results nicely

An **f-string** builds text with values dropped in. Put an **f** before the quotes, and variables inside **{ }**:

Python Code

```
price = 248.83  
f"Apple closed at {price}"
```

```
'Apple closed at 248.83'
```

Useful, but that number could be tidier. Next slide.

The two recipes we will reuse

A colon inside the braces sets the **format**:

Python Code

```
price = 248.83
r = 0.0231457
print(f"Price: {price:.2f}")
print(f"Return: {r:.2%}")
```

```
Price: 248.83
Return: 2.31%
```

- `:.2f` gives a plain number, 2 decimals
- `:.2%` gives a percentage, 2 decimals

These two cover almost everything we need all course.

True or False: booleans

A **comparison** asks a yes/no question. The answer is a **bool**:

Python Code

```
price = 248.83  
price > 250
```

False

Change **250** to **240** and run again.

Comparison operators

ask

write

equal?

`==` (two signs: a single `=` *assigns*)

not equal?

`!=`

greater / less?

`>` `<` `>=` `<=`

Python Code

```
ticker = "AAPL"  
ticker == "AAPL"
```

True

Booleans look modest now. Later they select rows of data, and later still they become the thing a model predicts.

Your turn 2

This cell runs with **no error**, and gives nonsense. Run it, look closely, then fix it so **total_cost** is a real number.

Python Code

```
price_text = "52.40"    # arrived from a file, as text
quantity = 4

total_cost = price_text * quantity
total_cost
```

What is `"52.40" * 4`? Then reach for `float(...)`.

6 • Functions and methods

PART 6

You have met `print` and `type`. Here are more, and the pattern behind them.

More functions

A **function** takes input in parentheses and returns a value. Two new ones:

Python Code

```
print(round(3.14159, 2))  
print(len("JPMorgan"))
```

3.14

8

round rounds to a number of decimals; **len** counts characters.

Functions we will use constantly

function	returns
<code>print(x)</code>	displays a value (<i>already using it</i>)
<code>type(x)</code>	the type of a value (<i>already using it</i>)
<code>round(x, n)</code>	<code>x</code> rounded to <code>n</code> decimals
<code>len(x)</code>	how many characters or items
<code>min(...)</code> <code>max(...)</code> <code>sum(...)</code>	smallest, largest, total

Methods: a function with a home

A **method** belongs to a value and is called with a **dot**:

Python Code

```
ticker = "aapl"  
ticker.upper()
```

```
'AAPL'
```

The pattern is `value.method(arguments)`.

One pattern, the whole course

That same dot-grammar runs everything ahead:

```
ticker.upper()      # today: text  
prices.sort()       # today: lists  
returns.mean()      # Session 3: data tables
```

Different objects, identical shape. Learn `value.method(...)` once, and later tools feel familiar. Try it live:

Python Code

```
"Machine Learning in Finance".count("in")
```

4

Your turn 3

Build exactly this text with an f-string: **Total return: 35.56%**

Python Code

```
jan = 183.56
dec = 248.83

total_return = ...
message = ...
message
```

First the return $(\text{dec} - \text{jan}) / \text{jan}$, then wrap it: `f"Total return: {...:.2%}"`.

7 • Lists

PART 7

The workhorse of the day, and the foundation for the data tables that come later.

One year is 252 numbers

Apple's closing price, first week of 2024, in a single **list**:

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]  
len(closes)
```

5

A list collects values, in order, under one name.

Why lists matter

One variable per day would be absurd: a year has about **252 trading days**. A list holds them all together.

Later in the course a data table will hold ten years of eleven stocks at once. It is built directly on this idea: many values, in order, under one name.

Indexing: pick one day

Positions count **from zero**: the index is the distance from the start.

index	0	1	2	3	4
	183.56	182.19	179.87	179.15	183.48
from the end	-5	-4	-3	-2	-1

Indexing in action

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]
print(closes[0])
print(closes[2])
```

```
183.56
179.87
```

`[0]` is the first trading day, `[2]` the third. Change the index and run.

Counting from the end

`[-1]` is the last item, whatever the list's length:

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]  
closes[-1]
```

183.48

`[-1]` is worth loving. You rarely know a list's exact length; you always know you want the most recent value.

Off the end

PREDICT

Five prices sit at positions 0 to 4. **What will asking for `closes[5]` give you?**
Commit to an answer, then run.

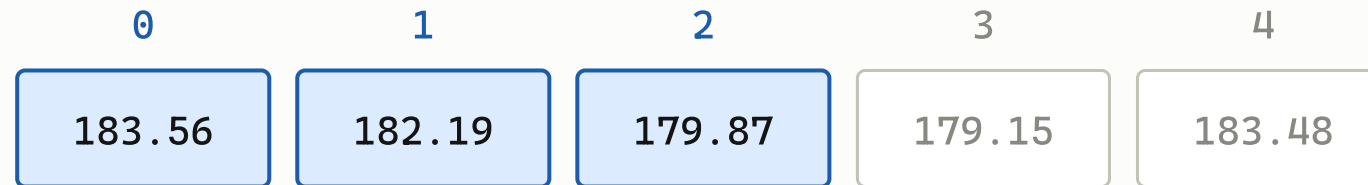
Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]  
closes[5]
```

Positions run 0 to 4, so what lives at position 5?

Slicing: pick a window

`list[start:stop]`: **start included, stop excluded.**



`closes[0:3]`

three items: $3 - 0 = 3$ · the stop index is not included

Slicing in action

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]  
closes[0:3]
```

```
[183.56, 182.19, 179.87]
```

Three items: positions 0, 1, 2. Position 3 is left out.

Open-ended slices

Leave an end blank to mean “from the start” or “to the end”:

Python Code

```
close = [183.56, 182.19, 179.87, 179.15, 183.48]  
print(close[:2])  
print(close[2:])
```

```
[183.56, 182.19]  
[179.87, 179.15, 183.48]
```

Lists can grow

`.append(...)` adds a value to the end:

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]
closes.append(185.56)
closes
```

```
[183.56, 182.19, 179.87, 179.15, 183.48, 185.56]
```

`.append` is a **method**: `closes.append(...)`, the dot-grammar again.

Where does a value sit?

`.index(...)` reports the **position** of a value:

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]  
closes.index(179.15)
```

3

It found **179.15** at position 3. We will use this to locate a stock's worst day.

Summaries over a whole list

The functions from earlier accept a list:

Python Code

```
close = [183.56, 182.19, 179.87, 179.15, 183.48]
print(min(close), max(close))
print(sum(close) / len(close))
```

```
179.15 183.56
181.65
```

`sum(...)` / `len(...)` is the average price of the week: your first statistic, from raw parts.

Your turn 4

From ten days, compute the **range** (highest minus lowest) of the **last five** days only.

Python Code

```
closes = [92.10, 93.45, 91.80, 94.20, 95.10,  
          96.30, 94.85, 97.60, 96.90, 98.25]  
  
last_five = ...  
day_range = ...  
day_range
```

Slice the last five first, then `max(...) - min(...)`. The answer is 3.40.

8 • Errors are information

PART 8

An error message is a diagnosis, not a scolding.

An error is a diagnosis

Run this deliberately broken cell and read what comes back:

Python Code

```
prices = [183.56, 182.19, 179.87]  
prices[10]
```

```
list index out of range
```

Nothing is damaged. Your other variables are fine. The runtime is fine.

Read it bottom-up

In Colab you get the **full traceback**. Read it from the bottom:

```
IndexError                                Traceback (most recent call last)
      1 prices = [183.56, 182.19, 179.87]
----> 2 prices[10]
```

IndexError: list index out of range

- the **last line** names the error type and what went wrong
- the arrow **---->** marks the exact line that failed

Read the last line, look at the arrow, fix that line, run again. That resolves most errors before you search for anything.

The four errors of week one

error	usually means	first thing to check
<code>NameError</code>	name never assigned	typo? case? did that cell run?
<code>TypeError</code>	operation meets the wrong type	is a “number” secretly text?
<code>SyntaxError</code>	Python cannot parse the line	missing quote, bracket, or comma
<code>IndexError</code>	that position does not exist	off by one? empty list?

Error playground

OPTIONAL

Each line has one classic mistake. Remove one `#` at a time, run, and read the message before fixing it.

Python Code

```
price = 100.5
ticker = "AAPL"
week = [1.2, -0.4, 0.8]

# price + 1
# ticker + price
# week[3]
```

Which of the four error types does each one produce?

9 • *A week of Apple, by hand*

PART 9

Everything from today, on real market data, and the course case begins.

The simple return

Yesterday's close p_0 , today's close p_1 . The **simple return** is

$$r = \frac{p_1 - p_0}{p_0}$$

the percentage change in price: what one unit invested earned in a day. It is the basic object of this whole course.

Deliberately familiar: the same percentage change you know from econometrics growth rates.

Apple's first week of 2024

Two lists kept in step, with a date and a price at each position:

Python Code

```
dates = ["Jan 02", "Jan 03", "Jan 04", "Jan 05", "Jan 08"]
closes = [183.56, 182.19, 179.87, 179.15, 183.48]
closes[0]
```

183.56

One day's return

From the first close to the second:

Python Code

```
closes = [183.56, 182.19, 179.87, 179.15, 183.48]  
(closes[1] - closes[0]) / closes[0]
```

```
-0.007463499673131426
```

Apple lost about three-quarters of a percent on 3 January, its second trading day of 2024. A return always belongs to the *later* of the two days.

Format it properly

The same return, as a clean percentage with the day attached:

Python Code

```
dates = ["Jan 02", "Jan 03", "Jan 04", "Jan 05", "Jan 08"]
closes = [183.56, 182.19, 179.87, 179.15, 183.48]
r1 = (closes[1] - closes[0]) / closes[0]
f"Return on {dates[1]}: {r1:.2%}"
```

```
'Return on Jan 03: -0.75%'
```

The whole week, combined

Return over the week, plus the lowest day, using indexing, `min`, and `.index` together:

Python Code

```
dates = ["Jan 02", "Jan 03", "Jan 04", "Jan 05", "Jan 08"]
closes = [183.56, 182.19, 179.87, 179.15, 183.48]
week_return = (closes[-1] - closes[0]) / closes[0]
lowest = min(closes)
low_day = dates[closes.index(lowest)]
f"Week: {week_return:.2%}, low of {lowest} on {low_day}"
```

```
'Week: -0.04%, low of 179.15 on Jan 05'
```

`closes.index(lowest)` finds *where* the minimum sits; `dates[...]` reads the same position in the other list.

Two lists, kept in step

Dates in one list, prices in another, connected only by position. In Session 3 a data **table** keeps them in one object, so they can never drift apart.

That single idea, many aligned columns in one structure, is what makes pandas, and everything after it, possible.

The year that week came from



252 trading days; our five are the shaded sliver. Doing a whole year by hand would be painful, which is why Session 3 brings in **pandas**.

The course case **CASE**

The Analyst's Notebook: one investigation, a part after most sessions, done at home.

You are a junior analyst building a **risk report** on a small universe of US stocks. The running question:

How much do these stocks move, how does that differ across them, and could that movement be **predicted** at all?

By Session 4 you will have framed that question precisely, with real data. From Session 5 on, you build the models that answer it.

The case, part by part

	data	your tools
Part 1 · today	Apple & Coca-Cola, 2024 daily closes	lists, indexing, returns
Part 2	the same, measured properly	functions, loops, dictionaries
Part 3	11 stocks, 2015–2024	pandas tables and plots
Part 4	the same, as a prediction problem	features, targets, evaluation
Parts 5 and 6	next month's volatility, forecast	cross-validation, ridge and lasso
Parts 8 and 9	a warning before a big move	classification, thresholds
Parts 11 and 12	the forecast and the warning again	trees, forests, boosting

No live downloads: every price ships with the course. Results are identical for everyone. Sessions 7, 10 and 13 are case lectures of their own, with no part.

Case Part 1: this week

`session_01_case.ipynb`, 10 questions, done at home before Session 2:

- load Apple's 252 closing prices of 2024 (*loader provided*)
- first close, last close, best day, worst day
- slice January and measure its return
- one broken cell to diagnose and fix
- the finish: the same measurements for Coca-Cola, and a first judgment: **which of the two moved more in 2024?**

Hints and full solutions are folded into the notebook. Work first, then check.

What stays with you

Runtime state

Cells share a memory; “correct” means correct top-to-bottom from a fresh start.

Types

`int`, `float`, `str`, `bool`; and `"150"` is not 150.

Functions & methods

`round(x, 2)` versus `ticker.upper()`: the dot-grammar runs the course.

Errors

Read the last line first. Nothing is broken.

Variables

`=` assigns a name to a value; assigning again overwrites.

Formatting

`f"{r:.2%}"` and `f"{p:.2f}"`: the two recipes we reuse.

Lists

Index from 0, `[-1]` is the end, slice `[a:b]` excludes `b`.

Simple return

`(p1 - p0) / p0`: the object this course is about.

Quick check

OPTIONAL

Without running anything, what does each line produce? (*a value, or which error?*)

```
a = [10, 20, 30]
```

```
a[3]                # 1
```

```
"5" + "5"          # 2
```

```
a[-1] > 25          # 3
```

```
share_count        # 4    (never assigned)
```

```
a[1:3]             # 5
```

Answers on the next slide; argue with your neighbour first.

Quick check: answers

	expression	result
1	<code>a[3]</code>	<code>IndexError</code> : positions are 0, 1, 2
2	<code>"5" + "5"</code>	<code>"55"</code> : strings glue
3	<code>a[-1] > 25</code>	<code>True</code> : 30 > 25
4	<code>share_count</code>	<code>NameError</code> : never assigned
5	<code>a[1:3]</code>	<code>[20, 30]</code> : stop excluded

Before the next session

HOMEWORK

Exercises

`session_01_exercises.ipynb`: 55 short tasks. Hints and solutions folded in.

HOMEWORK

Case Part 1

`session_01_case.ipynb`: the Apple versus Coca-Cola report.

IF STUCK

Get help

Ask a friend, ask an AI for a hint, or email me at jobo@econ.au.dk.
Stuck over 15 minutes? Note it and bring it next session.

Next session: functions, loops, and dictionaries, the tools that let you repeat a calculation over hundreds of prices at once instead of by hand.

Live: getting set up

NOW, TOGETHER

Opening Google Colab, saving your own copy, and running your first cells, live.